

Manual de laboratorio: SUMO

Erick Pérez P.

erickpatriciopp9@gmail.com

7 de abril de 2026

Tabla de contenidos

1 Introducción

- ▶ **Introducción**
- ▶ Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)
- ▶ Práctica 2: Simulación de tráfico vehicular con SUMO WebWizard
- ▶ Práctica 3: Estadísticas de movilidad vehicular en SUMO
- ▶ Práctica 4: Introducción a TraCI (Traffic Control Interface)
- ▶ Práctica 5: Clasificación de movilidad con aprendizaje automático
- ▶ Práctica 6: Aprendizaje automático aplicado a ITS

- Esta presentación tiene como finalidad ser una guía integral para una implementación de simulación de movilidad vehicular. El documento incluye una serie de prácticas destinadas a ser aplicadas con software de código abierto. Cada práctica proporciona los pasos necesarios para lograr los objetivos establecidos. En última instancia, el objetivo es establecer una simulación de una red vehicular implementado SUMO, TraCI y un modelo de aprendizaje automático para sistemas de transportación inteligente (ITSs).

SUMO

1 Introducción

- SUMO (Simulación de Movilidad Urbana), es un paquete de simulación de tráfico **multimodal** continuo, **microscópico**, altamente portátil y de código abierto diseñado para manejar grandes redes.
- SUMO es una suite de simulación de tráfico **gratuita** y de **código abierto**.
- Está disponible desde 2001 y permite modelar sistemas de tráfico, incluidos vehículos de carretera, transporte público y peatones.
- SUMO incluye una gran cantidad de herramientas de soporte que automatizan tareas para la **creación**, ejecución, **generación de rutas**, **visualización** entre otras.
- SUMO se puede mejorar con **modelos personalizados** y proporciona una **API para controlar** la simulación en línea (TraCI).

SUMO

1 Introducción

- SUMO se ha utilizado en varios proyectos para responder una gran variedad de preguntas de investigación:
 - Evaluación de los ciclos semafóricos.
 - Optimización de rutas.
 - Reducción de emisiones contaminantes.
 - Conducción autónoma.
 - Predicción y mejora de tráfico vehicular.
 - Evaluación de redes vehiculares (VANETs).
 - Manejo cooperativo (Platooning).
 - Seguridad vial y análisis de riesgos.
 - Planificación de rutas de transporte público.
 - Modelos de movilidad, emisiones, carga/descarga de batería de EVs.

1 Introducción

-

<https://tinyurl.com/267u5sdf>

Tabla de contenidos

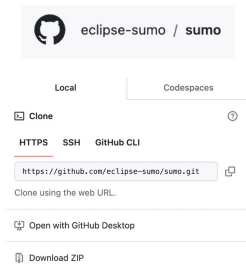
2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

- ▶ Introducción
- ▶ **Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)**
- ▶ Práctica 2: Simulación de tráfico vehicular con SUMO WebWizard
- ▶ Práctica 3: Estadísticas de movilidad vehicular en SUMO
- ▶ Práctica 4: Introducción a TraCI (Traffic Control Interface)
- ▶ Práctica 5: Clasificación de movilidad con aprendizaje automático
- ▶ Práctica 6: Aprendizaje automático aplicado a ITS

Instalación de SUMO

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

Binarios



Repositorios Linux



\$apt install sumo

Importar VM



 **sumo.ova** 

Docker



Docker commands

To push a new tag to this repository:

```
docker push pablobarbecho/sem:tagname
```

[Public View](#)

<https://drive.google.com/drive/folders/1hui6-4tgCHItcgMaaGh6zg06ZQtv5aLJ?usp=sharing>

Instalación de SUMO mediante archivos binarios

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

Objetivos:

- Utilizar los archivos binarios para instalar el simulador sobre el sistema operativo Linux (Guía realizada sobre - Ubuntu 22 LTS).
- Conocer los archivos necesarios para correr una simulación sobre SUMO.
- Lanzar una simulación desde la línea de comandos (CLI) y desde la interfaz gráfica (GUI).

Materiales:

- **Ordenador** con sistema operativo Linux (Ubuntu 22 LTS).
- Documentación
SUMO:https://sumo.dlr.de/docs/Installing/Linux_Build.html

Instalación de SUMO mediante archivos binarios

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

Instrucciones:

1. Actualizar los paquetes del sistema operativo.

```
sudo apt-get update
```

2. Instalar las herramientas y librerías necesarias para el funcionamiento de SUMO. Estas herramientas son:

```
sudo apt-get install git cmake python3 g++ libxerces-c-dev libfox  
-1.6-dev libgdal-dev libproj-dev libgl2ps-dev python3-dev swig  
default-jdk maven libeigen3-dev
```

```
sudo apt-get install ccache libavformat-dev libswscale-dev  
libopengl-dev python3-pip python3-setuptools
```

```
sudo apt-get install libgtest-dev gettext tkdiff xvfb flake8 astyle  
python3-autopep8 pip3 install texttest
```

```
10/122 sudo apt-get install python3-pandas python3-rtree python3-pyproj
```

Instalación de SUMO mediante archivos binarios

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

```
sudo apt-get install python3-pyproj python3-rtree python3-pandas  
python3-flake8 python3-autopep8 python3-scipy python3-pulp python3-  
-ezdxf
```

3. Una vez instaladas las librerías necesarias, es momento de obtener el código fuente de SUMO.

```
git clone --recursive https://github.com/eclipse-sumo/sumo
```

4. Luego, dentro de la carpeta descargada, es necesario realizar la captura adicional de los reemplazos para obtener un historial del proyecto local. Esto se logra con los siguientes comandos.

Instalación de SUMO mediante archivos binarios

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

```
cd sumo  
git fetch origin refs/replace/*:refs/replace/*
```

5. Luego instale las piezas restantes utilizando pip. Es necesario estar ubicados dentro de la carpeta sumo que se ha descargado previamente, con el fin de llamar al archivo `requirements.txt` ubicado dentro de la carpeta de sumo.

```
pip install -r tools/requirements.txt
```

Para instalar las dependencias más comunes con su gestor de paquetes en ubuntu, puede utilizar:

```
sudo apt-get install python3-pandas python3-rtree python3-pyproj
```


Instalación de SUMO mediante archivos binarios

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

6. Antes de compilar es recomendable definir la variable de entorno SUMO_HOME. El comando a utilizar es el siguiente:

```
export SUMO_HOME="/home/<user>/sumo-<version>"
```

7. Otra opción es modificar los archivos: /etc/bash.bashrc y /home/<user>/ .bashrc

```
GNU nano 6.2 /etc/bash.bashrc
See "man sudo_root" for details.
EOF
fi
esac
fi

# if the command-not-found package is installed, use it
if [ -x /usr/lib/command-not-found -o -x /usr/share/command-not-found/command-not-found ]
function command_not_found_handle {
    # check because c-n-f could've been removed in the meantime
    if [ -x /usr/lib/command-not-found ]; then
        /usr/lib/command-not-found -- "$1"
        return $?
    elif [ -x /usr/share/command-not-found/command-not-found ]; then
        /usr/share/command-not-found/command-not-found -- "$1"
        return $?
    else
        printf "%s: command not found\n" "$1" >&2
        return 127
    fi
}
fi

export SUMO_HOME="/home/sumo/sumo"
```

```
GNU nano 6.2 /home/sumo/.bashrc
# Add on "alert" alias for long running commands. Use like so:
# sleep 10; alert
alias alert='notify-send --urgency=low -i "${[ $? = 0 ]} && echo terminal || echo error"'

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
    if [ -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi

export SUMO_HOME="/home/sumo/sumo"
```

Instalación de SUMO mediante archivos binarios

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

8. Finalmente, es necesario construir los binarios SUMO con cmake. El siguiente comando construye los binarios de SUMO:

```
cmake -B build .
```

Después de que esto haya terminado, es necesario correr el siguiente comando:

```
cmake --build build -j $(nproc)
```

El comando `nproc` le da el número de núcleos lógicos en su ordenador, para que esa marca se inicie de forma paralela construir puestos de trabajo que lo haga construir mucho más rápido.

9. Instalar SUMO y actualizar variable de entorno.

```
sudo cmake --install build  
export SUMO_HOME=/usr/local/share/sumo
```

Instalación de SUMO - Shell Script (Github)

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

1. Descargar o clonar proyecto de Github:
`https://github.com/erickperez9/ManualSUMO.git`.
2. Descomprimir carpeta en cualquier directorio (e.g., Desktop).
3. Ingresar en la carpeta ManualSUMO y dar permisos de ejecución al shell script
`install-sumo.sh`

```
$chmod +x Desktop/ManualSUMO-main/install-sumo.sh
```

4. Cruzar los dedos para que no salte un error.
5. Ejecutar el script `install-sumo.sh`.

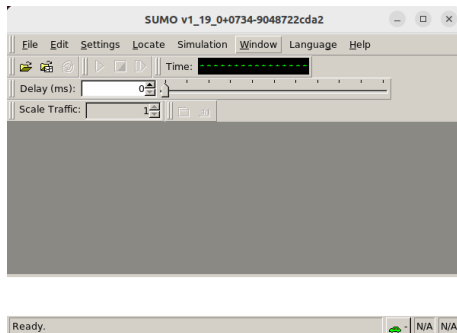
```
$./install-sumo.sh
```

Instalación de SUMO - Shell Script (Github)

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

6. Para correr SUMO abrimos una terminal de Linux y tenemos dos opciones: `sumo` para correr por consola (CLI) o `sumo-gui` correr con interfaz gráfica (GUI).

```
sumo@vm: ~$ sumo
Eclipse SUMO v1_19_0+1213-edb5da6ce7a
Build features: Linux-6.5.0-25-generic aarch64 GNU 11.4.0 Rel
Intl SWIG GDAL FFmpeg OSG GL2PS Eigen
Copyright (C) 2001-2024 German Aerospace Center (DLR) and oth
dlr.de
License EPL-2.0: Eclipse Public License Version 2 <https://ec
l-v20.html>
Use --help to get the list of options.
sumo@vm: ~$
```



Generación de archivos de simulación de SUMO

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

Objetivos

- Conocer los archivos necesarios para correr una simulación sobre SUMO.
- Simular una red vehicular sobre SUMO desde la línea de comandos.
- Simular una red vehicular sobre SUMO desde la interfaz gráfica.

Materiales

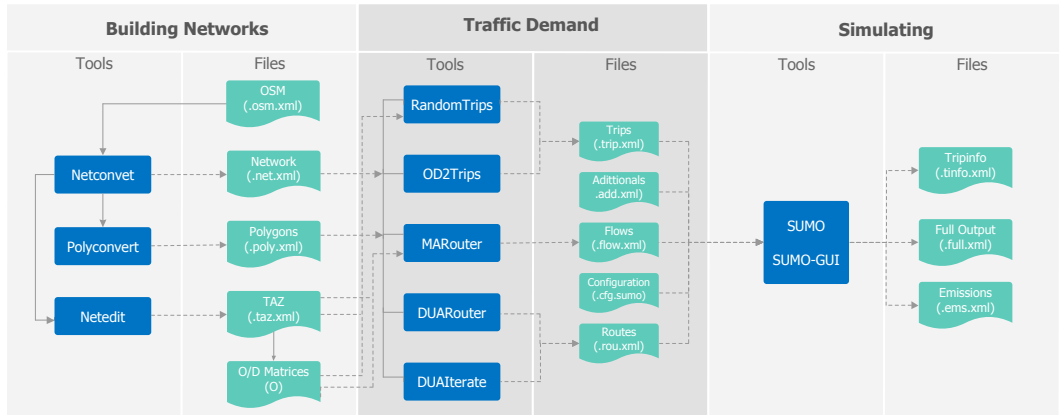
- **Ordenador** con sistema operativo Linux (Ubuntu 22 LTS).
- **SUMO** instalado.

Generación de archivos de simulación de SUMO

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

Introducción

- Flujo de trabajo de SUMO

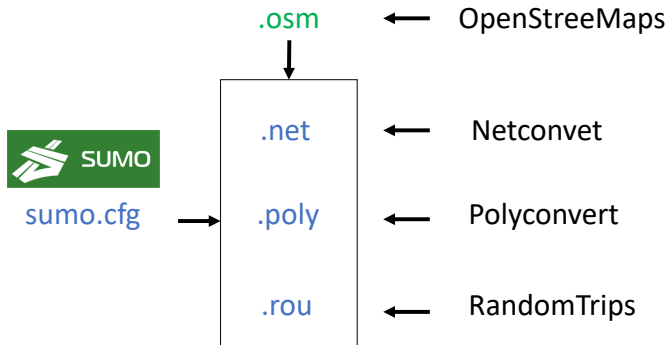


Generación de archivos de simulación de SUMO

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

Introducción

- Archivos necesarios para una simulación:



Generación de archivos de simulación de SUMO

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

Introducción

- Herramientas de generación de tráfico.

Tool	Inputs	Outputs	Traffic assignment method
OD2Trips (OD2)	Network, O/D Matrices, Additional	Trips	Incremental
DUARouter (DUAR)	Network, Trips, Additional	Routes	Empty-network
MARouter (MAR)	Network, O/D Matrices, Additional	Flows	Incremental
RandomTrips (RT)	Network, Additional	Trips	Incremental
DUALterate (DUAL)	Network, Trips, Additional	Routes	Iterative

Pablo Barbecho Bautista, Luis Urquiza Aguiar, Mónica Aguilar Igartua, "How does the traffic behavior change by using SUMO traffic generation tools", Computer Communications, 2021, ISSN 0140-3664, September 2021, (IF 2020 = 3.167; 41/91; Telecommunications; Q2), DOI:<https://doi.org/10.1016/j.comcom.2021.09.023>

Generación de archivos de simulación de SUMO

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

Introducción

- Herramientas de generación de tráfico.

Comparison of the main features of SUMO traffic demand generation tools. O/D means (Origen/Destination)

	Characteristics	OD2	DUAR	MAR	RT	DUAI
Routes Generation	Use the Dijkstra algorithm	✗	✓	✗	✗	✓
	Use detectors	✓	✗	✗	✗	✗
	Use O/D matrices	✓	✗	✓	✗	✗
	O/D points are mandatory	✓	✓	✓	✗	✓
	Full traffic files	✗	✓	✓	✗	✓
Traffic Demand Support	> 3 vehicle types	✓	✓	✓	✓	✓
	Buss stops for vehicle flows	✗	✓	✗	✓	✓
	TAZ districts	✓	✗	✓	✗	✗
Efficient Scenarios	Urban areas	✓	✓	✓	✓	✓
	Rural areas	✗	✓	✗	✓	✓
Avoid	Congestion situations	✓	✗	✓	✓	✓
	Deadlock network	✓	✗	✗	✓	✓

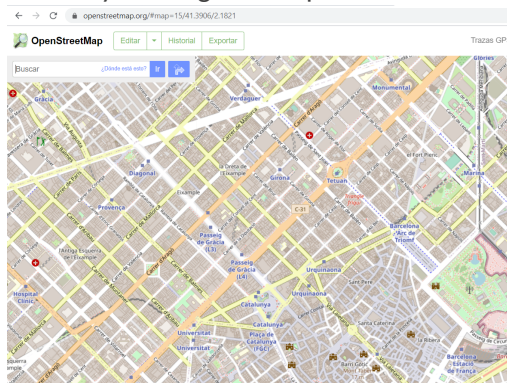
Pablo Barbecho Bautista, Luis Urquiza Aguiar, Mónica Aguilar Igartua, "How does the traffic behavior change by using SUMO traffic generation tools", Computer Communications, 2021, ISSN 0140-3664, September 2021, (IF 2020 = 3.167; 41/91; Telecommunications; Q2), DOI:<https://doi.org/10.1016/j.comcom.2021.09.023>

Generación de archivos de simulación de SUMO

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

Instrucciones:

1. Acceder al sitio web de OSM y descargar el mapa.

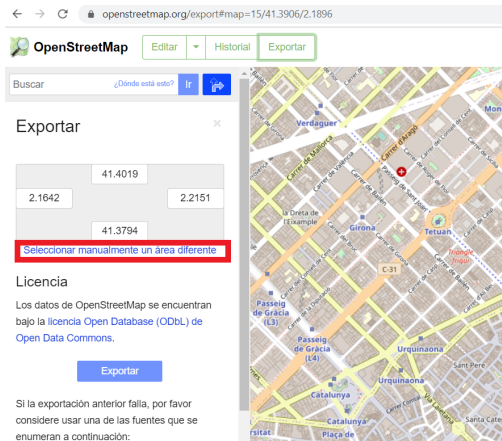


<https://www.openstreetmap.org/>

Generación de archivos de simulación de SUMO

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

- Haga clic en "Exportar" y luego seleccione la opción: "Seleccionar manualmente un área diferente".



Generación de archivos de simulación de SUMO

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

3. Seleccionar cualquier área del mapa y presionar el botón "Exportar".

Exportar

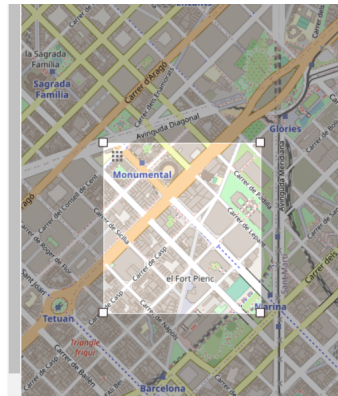
	41.4012	
2.1779		2.1860
	41.3946	

Licencia

Los datos de OpenStreetMap se encuentran bajo la [licencia Open Database \(ODbL\)](#) de [Open Data Commons](#).

Exportar

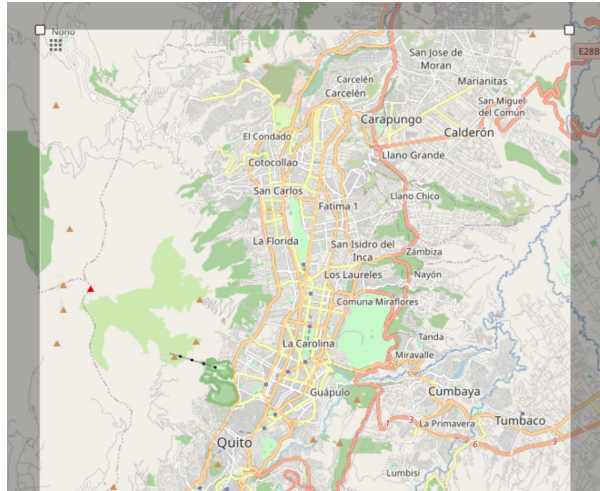
Si la exportación anterior falla, por favor considere usar una de las fuentes que se enumeran a continuación:



Generación de archivos de simulación de SUMO

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

3. Seleccionar cualquier área del mapa y presionar el botón "Exportar". Pruebe exportar el mapa de todo Quito. **Nota:** máximo es 50000 nodos.



Generación de archivos de simulación de SUMO

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

4. Guardar el archivo del mapa en formato `mapa.osm`.
5. Ahora, invocar la herramienta *netconvert* para generar la red de carreteras correspondiente a un escenario urbano (verificar comandos en **Github** Lab_1):

```
netconvert -v --ramps.guess --remove-edges.isolated --edges.join --  
geometry.remove --osm-files mapa.osm -o mapa.net.xml
```

6. Luego se invoca la herramienta *polyconvert*:

```
polyconvert -v --net-file mapa.net.xml --osm-files mapa.osm -o mapa.  
poly.xml
```

Github: <https://github.com/erickperez9/ManualSUMO.git>

Generación de archivos de simulación de SUMO

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

7. El siguiente paso es generar es el archivo de rutas. Usamos la herramienta *Random-Trips* localizada dentro del directorio `/opt/sumo/tools/`. Para generar este archivo se debe invocar el script de Python `randomTrips.py`.

```
python3 /opt/sumo/tools/randomTrips.py -v -n mapa.net.xml -r mapa.rou.xml
```

Generación de archivos de simulación de SUMO

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

Una opción de utilidad es `-p`. Esta opción permite generar tráfico en el que consten n vehículos distribuidos uniformemente en un período de tiempo. `-p` se obtiene de:

$$p = \frac{t_1 - t_0}{n} \quad (1)$$

Donde, t_1 y t_0 son el tiempo final e inicial de la simulación, respectivamente. Por ejemplo, 3600 segundos, comenzando en el segundo 0.

$$p = \frac{3600}{100} = 36 \quad (2)$$

```
python3 /opt/sumo/tools/randomTrips.py -v -b 0 -e 3600 -p 36 -n mapa.net.xml -r mapa.rou.xml
```


Generación de archivos de simulación de SUMO

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

8. Finalmente, es necesario crear el archivo de configuración del escenario:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <input>
    <net-file value="mapa.net.xml"/>
    <route-files value="mapa.rou.xml"/>
    <additional-files value="mapa.poly.xml"/>
  </input>
  <processing>
    <ignore-route-errors value="true"/>
  </processing>
  <time>
    <begin value="0"/>
    <end value="3600"/>
  </time>
</configuration>
```

Simulación de tráfico vehicular en SUMO (CLI)

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

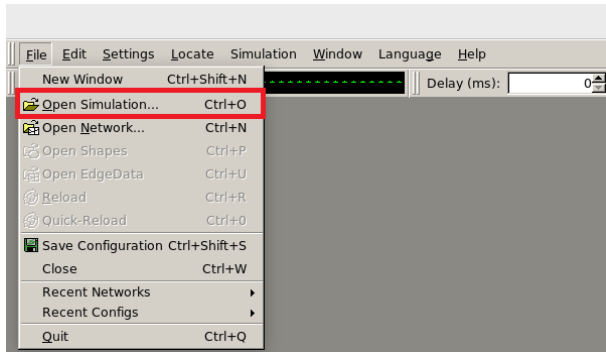
9. Abrir una consola de Linux.
10. Es necesario indicar la ubicación al archivo de configuración del escenario con extensión `sumo.cfg`:

```
$sumo -v -c mapa.sumo.cfg
```

Simulación de tráfico vehicular en SUMO (GUI)

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

11. Abrir una consola de Linux y ejecutar `sumo-gui`
12. En la SUMO GUI cargamos el archivo de configuración del escenario `.sumo.cfg`.
13. Verificar errores o advertencias en la consola de la GUI.



Simulación de tráfico vehicular en SUMO

2 Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)

14. Finalmente, para correr la simulación se debe tener en cuenta el retardo de la misma. Esto se configura en la parte superior (recuadro rojo) del simulador.
15. Dar clic en la opción *Run* (recuadro azul) y la simulación estará en marcha.
16. Verificar errores o advertencias en la consola de la GUI.

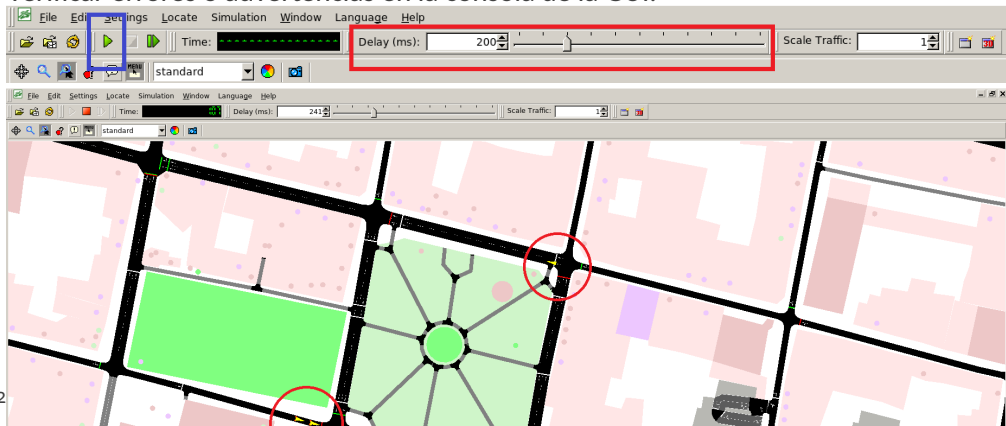


Tabla de contenidos

3 Práctica 2: Simulación de tráfico vehicular con SUMO WebWizard

- ▶ Introducción
- ▶ Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)
- ▶ **Práctica 2: Simulación de tráfico vehicular con SUMO WebWizard**
- ▶ Práctica 3: Estadísticas de movilidad vehicular en SUMO
- ▶ Práctica 4: Introducción a TraCI (Traffic Control Interface)
- ▶ Práctica 5: Clasificación de movilidad con aprendizaje automático
- ▶ Práctica 6: Aprendizaje automático aplicado a ITS

Simulación de tráfico vehicular con SUMO WebWizard

3 Práctica 2: Simulación de tráfico vehicular con SUMO WebWizard

Objetivos

- Generar los archivos necesarios para correr una simulación sobre SUMO con la herramienta WebWizard.
- Simular una red vehicular sobre SUMO.

Materiales

- **Ordenador** con sistema operativo Linux (Ubuntu 22 LTS).
- **SUMO** instalado.

Simulación de tráfico vehicular con SUMO WebWizard

3 Práctica 2: Simulación de tráfico vehicular con SUMO WebWizard

Introducción

- Dentro del ecosistema de SUMO, una **herramienta particularmente útil** y accesible es el OSMWebWizard.
- OSMWebWizard **simplifica el proceso de importación** de datos de OpenStreetMap en formatos compatibles con SUMO, permitiendo a los usuarios generar rápidamente simulaciones.
- Esta herramienta **ofrece una interfaz gráfica intuitiva** que guía a los usuarios a través de los pasos necesarios para importar mapas y configurar parámetros de simulación.

Simulación de tráfico vehicular con SUMO WebWizard

3 Práctica 2: Simulación de tráfico vehicular con SUMO WebWizard

Instrucciones

1. Comenzamos por instalar Chrome en Ubuntu. Se ha preparado un script que facilita la instalación. Referirse a <https://github.com/erickperez9/ManualSUMO.git> a la ubicación Lab_2 script `chrome_install.sh`
2. Descargamos el script y agregamos permisos de ejecución (Ubicarse en el directorio de descarga):

```
$chmod +x chrome_install.sh
```

3. Ejecutamos el script `chrome_install.sh`

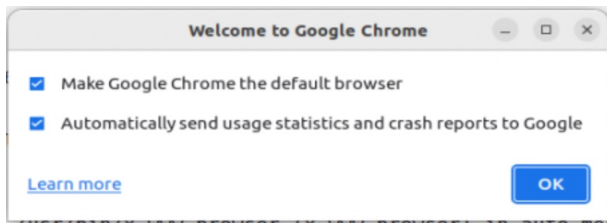
```
$/chrome_install.sh
```


Simulación de tráfico vehicular con SUMO WebWizard

3 Práctica 2: Simulación de tráfico vehicular con SUMO WebWizard

Instrucciones

4. Buscamos Chrome en el buscador de Ubuntu y abrimos la aplicación. Marcamos como explorador por defecto.



Simulación de tráfico vehicular con SUMO WebWizard

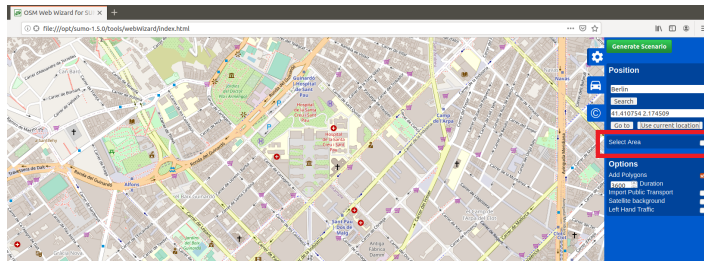
3 Práctica 2: Simulación de tráfico vehicular con SUMO WebWizard

Instrucciones

5. Invoque la herramienta: `osmWebWizard.py`. La herramienta está en la ruta: `/opt/sumo/tools`.

```
$python3 /opt/sumo/tools/osmWebWizard.py
```

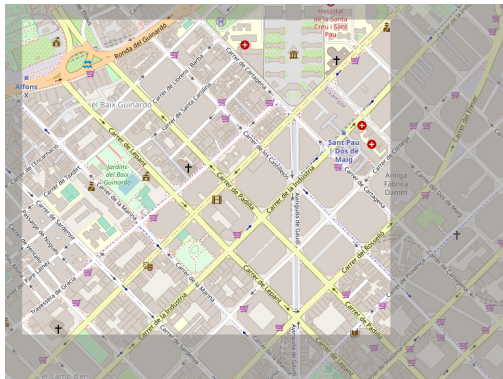
6. Se abrirá un navegador web con el mapa de osm. Se puede modificar la ubicación y luego se debe seleccionar la opción *Seleccionar Área*.



Simulación de tráfico vehicular con SUMO WebWizard

3 Práctica 2: Simulación de tráfico vehicular con SUMO WebWizard

7. Ahora se puede seleccionar un área y exportar el mapa. **Nota:** no tiene límite de nodos.
8. Verifique la consola mientras se genera el escenario.
9. Una vez generado el escenario, se abre automáticamente SUMO GUI y se ejecuta la simulación. Para terminar puede parar el proceso con `Ctrl+C` en la consola.



Simulación de tráfico vehicular con SUMO WebWizard

3 Práctica 2: Simulación de tráfico vehicular con SUMO WebWizard

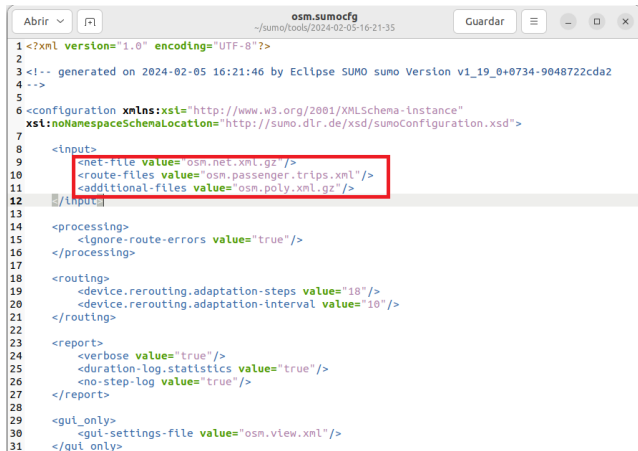
10. Una vez finalizado el proceso, se agrega una carpeta dentro del directorio desde donde ejecutó osmWebWizard, con la fecha en la que se generaron los archivos como nombre, por ejemplo:

```
sumo@sumo-VirtualBox:~/sumo/tools/2024-02-05-16-21-35$ ls -l
total 1824
-rwxr-xr-x 1 sumo sumo    573 feb  5 16:21 build.bat
-rw-rw-r-- 1 sumo sumo 813050 feb  5 16:21 osm_bbox.osm.xml.gz
-rw-rw-r-- 1 sumo sumo   1148 feb  5 16:21 osm.netccfg
-rw-rw-r-- 1 sumo sumo 724502 feb  5 16:21 osm.net.xml.gz
-rw-rw-r-- 1 sumo sumo  48588 feb  5 16:21 osm.passenger.trips.xml
-rw-rw-r-- 1 sumo sumo    690 feb  5 16:21 osm.polycfg
-rw-rw-r-- 1 sumo sumo 252912 feb  5 16:21 osm.poly.xml.gz
-rw-rw-r-- 1 sumo sumo    929 feb  5 16:21 osm.sumocfg
-rw-rw-r-- 1 sumo sumo    88 feb  5 16:21 osm.view.xml
-rwxr-xr-x 1 sumo sumo    23 feb  5 16:21 run.bat
sumo@sumo-VirtualBox:~/sumo/tools/2024-02-05-16-21-35$
```

Simulación de tráfico vehicular con SUMO WebWizard

3 Práctica 2: Simulación de tráfico vehicular con SUMO WebWizard

El archivo más importante siempre será el archivo de configuración, en este caso tiene como nombre `osm.sumocfg`. En este archivo se encuentran la configuración de entrada, procesamiento y salida de la simulación.



```
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <!-- generated on 2024-02-05 16:21:46 by Eclipse SUMO sumo Version v1_19_0+0734-9048722cda2
4 -->
5
6 <configuration xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
7   xsi:noNamespaceSchemaLocation="http://sumo.dlr.de/xsd/sumoConfiguration.xsd">
8
9   <input>
10     <net-file value="osm.net.xml.gz" />
11     <route-files value="osm.passenger.trips.xml" />
12     <additional-files value="osm.poly.xml.gz" />
13   </input>
14
15   <processing>
16     <ignore-route-errors value="true" />
17   </processing>
18
19   <routing>
20     <device.rerouting.adaptation-steps value="18" />
21     <device.rerouting.adaptation-interval value="10" />
22   </routing>
23
24   <report>
25     <verbose value="true" />
26     <duration-log.statistics value="true" />
27     <no-step-log value="true" />
28   </report>
29
30   <gui_only>
31     <gui-settings-file value="osm.view.xml" />
32   </gui_only>
33 </configuration>
```

Simulación de tráfico vehicular con SUMO WebWizard

3 Práctica 2: Simulación de tráfico vehicular con SUMO WebWizard

11. Compare el archivo `sumocfg` generado con `osmWebWizard.py` con el archivo que usamos en la práctica anterior `sumo.cfg` descargado de Github. Qué diferencias puede notar?
12. Genere un nuevo archivo `sumo.cfg` con la estructura base para el resto de prácticas.

Tabla de contenidos

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

- ▶ Introducción
- ▶ Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)
- ▶ Práctica 2: Simulación de tráfico vehicular con SUMO WebWizard
- ▶ **Práctica 3: Estadísticas de movilidad vehicular en SUMO**
- ▶ Práctica 4: Introducción a TraCI (Traffic Control Interface)
- ▶ Práctica 5: Clasificación de movilidad con aprendizaje automático
- ▶ Práctica 6: Aprendizaje automático aplicado a ITS

Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

Objetivos

- Generar archivos de salida (outputs) de la simulación en SUMO.
- Entender el contenido de los archivos de salida de las simulaciones.
- Analizar los datos de las simulaciones con scripts en Python.

Materiales

- **Ordenador** con sistema operativo Linux (Ubuntu 22 LTS).
- **SUMO** instalado.

Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

Introducción

- SUMO permite generar una **gran cantidad de mediciones** diferentes.
- Todos los valores que se recopilan en los archivos de salida tienen una **estructura común** (XML).
- De forma predeterminada, todos los **outputs están deshabilitados** y deben activarse en el archivo de configuración del escenario (sumo.cfg).
- Algunas de las salidas disponibles deben definirse dentro de **archivos adicionales**.
- Se incluye la herramienta (script) **xml2csv.py** que permite convertir cualquier output a un formato de archivo plano (CSV).

Documentación: <https://sumo.dlr.de/docs/Simulation/Output/index.html>

Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

Instrucciones:

1. El primer paso es generar los archivos necesarios para la simulación (verificar prácticas 2,3).
2. Luego, es necesario modificar el archivo de configuración de SUMO `.sumo.cfg`. En este archivo es necesario especificar las salidas que se requieren, además de identificar el nombre (ruta) del archivo de salida.

```
<output >  
    <tripinfo -output value="tripinfo.xml"/>  
    <fcd-output value="fcd.xml"/>  
    <amitran-output value="trajectories.xml"/>  
</output >
```

```
<emissions >  
    <device.emissions.probability value="1"/>  
</emissions >
```

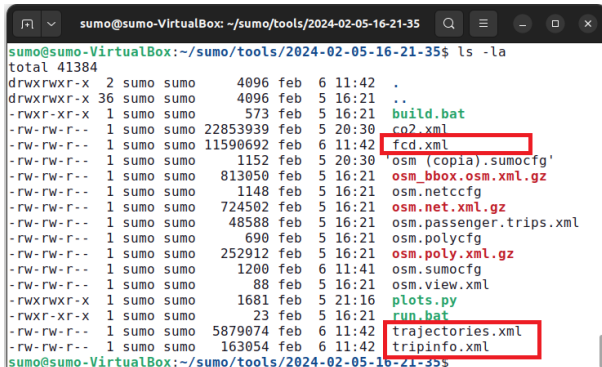
Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

3. Luego, correr la simulación de SUMO.

```
sumo -c osm.sumocfg
```

4. En la ruta definida se encontrarán los archivos de salida generados.



```
sumo@sumo-VirtualBox: ~/sumo/tools/2024-02-05-16-21-35$ ls -la
total 41384
drwxrwxr-x  2 sumo sumo    4096 feb  6 11:42 .
drwxrwxr-x 36 sumo sumo    4096 feb  5 16:21 ..
-rwxr-xr-x  1 sumo sumo     573 feb  5 16:21 build.bat
-rw-rw-r--  1 sumo sumo 22853939 feb  5 20:30 co2.xml
-rw-rw-r--  1 sumo sumo 11590692 feb  6 11:42 fcd.xml
-rw-rw-r--  1 sumo sumo    1152 feb  5 20:30 'osm (copia).sumocfg'
-rw-rw-r--  1 sumo sumo   813050 feb  5 16:21 osm_bbox.osm.xml.gz
-rw-rw-r--  1 sumo sumo    1148 feb  5 16:21 osm.netccfg
-rw-rw-r--  1 sumo sumo   724502 feb  5 16:21 osm.net.xml.gz
-rw-rw-r--  1 sumo sumo   485888 feb  5 16:21 osm.passenger.trips.xml
-rw-rw-r--  1 sumo sumo     690 feb  5 16:21 osm.polycfg
-rw-rw-r--  1 sumo sumo   252912 feb  5 16:21 osm.poly.xml.gz
-rw-rw-r--  1 sumo sumo    1200 feb  6 11:41 osm.sumocfg
-rw-rw-r--  1 sumo sumo      88 feb  5 16:21 osm.view.xml
-rwxrwxr-x  1 sumo sumo    1681 feb  5 21:16 plots.py
-rwxr-xr-x  1 sumo sumo      23 feb  5 16:21 run.bat
-rw-rw-r--  1 sumo sumo 5879074 feb  6 11:42 trajectories.xml
-rw-rw-r--  1 sumo sumo 163054 feb  6 11:42 tripinfo.xml
sumo@sumo-VirtualBox:~/sumo/tools/2024-02-05-16-21-35$
```

Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

5. Para facilitar el manejo de las estadísticas, vamos a convertir los outputs en formato XML en un formato en texto plano CSV. Para esto usamos la herramienta `xml2csv.py` incluida en la suite de SUMO dentro del directorio `/opt/sumo/tools/xml/`.

```
$python3 /opt/sumo/tools/xml/xml2csv.py -s , tripinfo.xml
```

El archivo resultante `tripinfo.csv` se ubicará en la ubicación desde donde ejecutamos la herramienta.

Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

6. Con los outputs obtenidos se pueden graficar estadísticas generales. Por ejemplo, en el repositorio de Github <https://github.com/erickperez9/ManualSUMO.git>, se presenta un script en python (`plots-v1-1.py`) dentro de la ubicación:
`Scripts plots/plots-v1-1.py`.

El script permite graficar los valores de: `tripinfo_routeLength`, `emissions_CO2_abs` y `tripinfo_duration`, respecto a `tripinfo_id`. Se requiere el archivo `tripinfo.csv` como elemento de entrada del script.

Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

7. Hemos de verificar el script de python (`plots-v1-1.py`) para conocer las opciones generales.
8. Ejecutamos el script con parámetro de entrada (`-i`) el archivo `tripinfo.csv`:

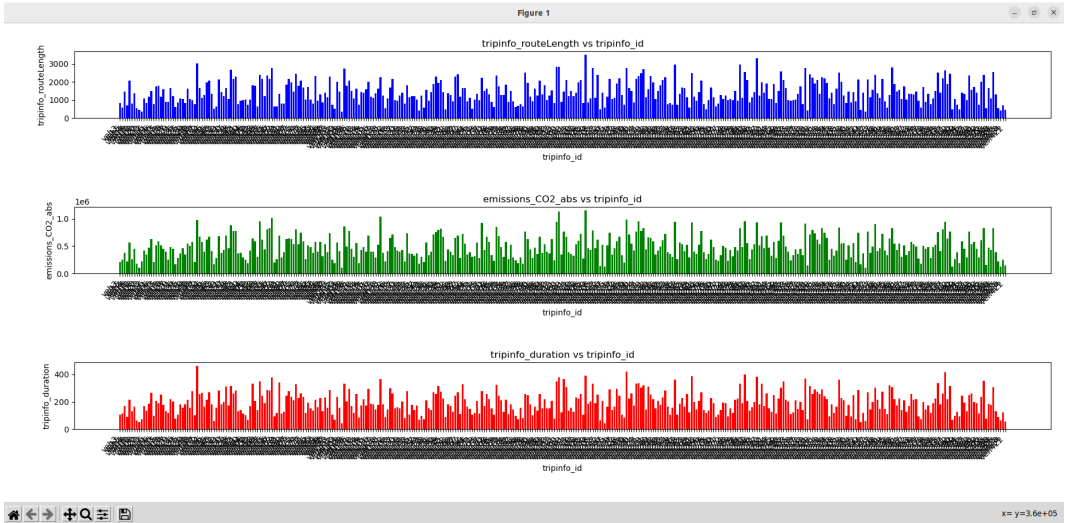
```
$python3 plots-v1-1.py -i tripinfo.csv
```

9. En el caso de un error de compatibilidad de versiones de paquetes de Python se puede actualizar según corresponda con:

```
$pip3 install --upgrade numexpr  
$pip3 install --upgrade bottleneck
```

Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO



Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

8. El código presentado se puede generalizar para que mediante línea de comandos el usuario deba ingresar el archivo `csv`, y las columnas de este archivo que desea graficar (sin espacios). En el repositorio de Github se presenta un ejemplo (`plots-v1-4.py`) para generalizar los plots, teniendo en cuenta que se graficará la columna seleccionada versus la columna `tripinfo_id`.

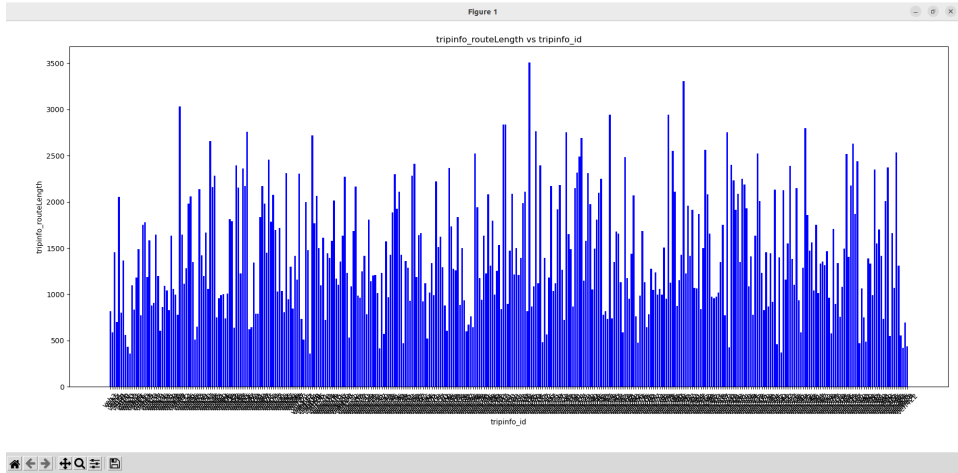
```
python3 plots-v1-4.py -i tripinfo.csv --columns tripinfo_routeLength
```

```
python3 plots-v1-4.py -i tripinfo.csv --columns tripinfo_routeLength  
    ,emissions_CO2_abs
```

```
python3 plots-v1-4.py -i tripinfo.csv --columns tripinfo_routeLength  
    ,emissions_CO2_abs ,tripinfo_duration
```

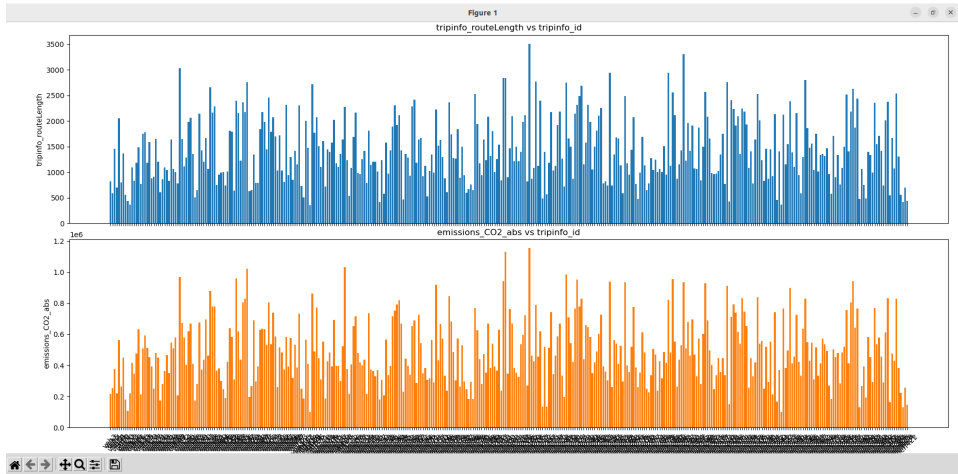

Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO



Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

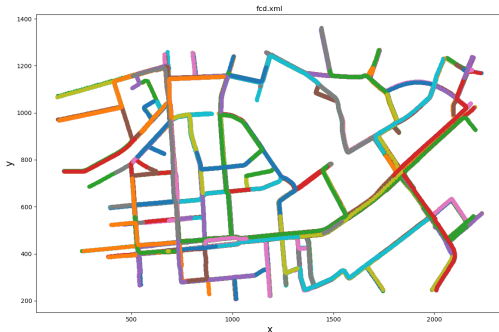


Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

9. Dentro de las herramientas de SUMO existen otras herramientas que permiten graficar ciertos aspectos de la simulación.
10. **Trayectorias:**

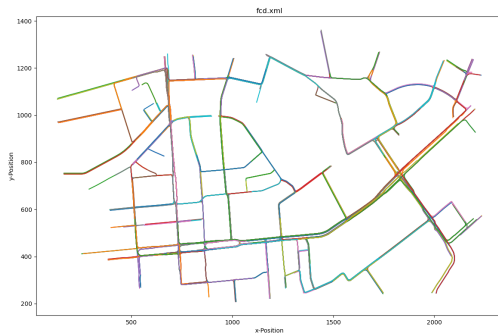
```
python3 /opt/sumo/tools/visualization/plotXMLAttributes.py -x x -y y  
-s 1 -o allXY_output.png fcd.xml --scatterplot
```



Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

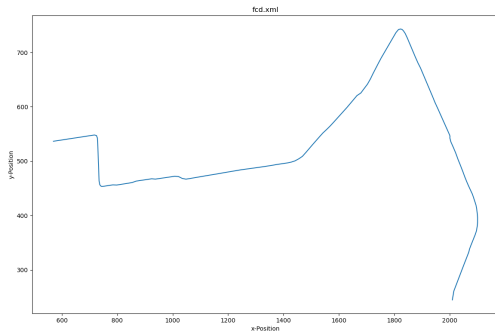
```
python3 /opt/sumo/tools/plot_trajectories.py -t xy -o  
allLocations_output.png fcd.xml
```



Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

11. Además, es posible seleccionar y graficar la trayectoria de uno o más vehículos. Para esto se requiere de aplicar un filtro al script `plot_trajectories.py`, con el parámetro `-filter-ids` el cual indica el ID del vehículo seleccionado. Si se requiere filtrar un vehículo adicional, basta con colocar el ID del otro vehículo seguido de una coma.

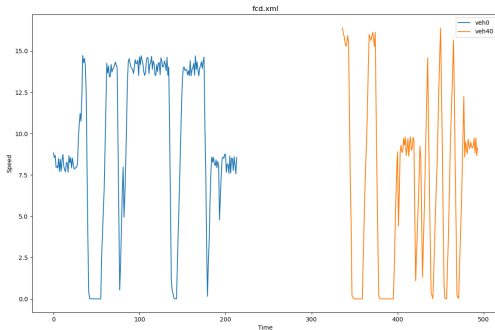


Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

12. **Velocidad:** También es posible graficar y comparar la velocidad de los vehículos en la simulación de SUMO. Para lograr esto se hace uso del script `plot_trajectories.py` ubicado en `/tools`. A continuación se muestra un ejemplo de como usarlo:

```
python /opt/sumo/tools/plot_trajectories.py -t ts -o name.png fcd.xml  
--filter -ids 13,26,45
```



Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

13. **Densidad de vehículos:** Otra de las herramientas que brinda SUMO es el script `plot_net_dump.py` ubicado en `tools/visualization/`. Este script muestra una red, donde los colores y el ancho de los bordes de la red se establecen en dependencia de los atributos de borde definidos.

Las medidas de densidad se localizan en el archivo `edge.data.xml` (https://sumo.dlr.de/docs/Simulation/Output/Lane-_or_Edge-based_Traffic_Measures.html). Este archivo contiene datos de cada carretera como la ocupación o densidad (veh/km) en cada instante de la simulación.

Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

Note que no hemos generado este output anteriormente. Bastaría con agregar el tópico respectivo (`edgedata-output`) en los outputs del archivo `sumo.cfg`

```
<output>  
  <emission-output value="co2.xml"/>  
  <tripinfo-output value="tripinfo.xml"/>  
  <fcd-output value="fcd.xml"/>  
  <edgedata-output value="edgedata.xml"/>  
</output>
```

El archivo resultante `edgedata.xml` almacena entre otros datos la densidad de vehículos en cada carretera *veh/km*.

Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

Los pesos de los bordes se leen a partir de archivo "edgedumps", tales como: *edgelane traffic*, *edgelane emissions*, o *edgelane noise*.

```
python3 /opt/sumo/tools/visualization/plot_net_dump.py -v -n osm.net.xml --measures density,density --xlabel [m] --ylabel [m] --default-width 1 -i edgedata.xml,edgedata.xml --xlim 000,2600 --ylim 000,1500 --default-width .5 --min-color-value 0 --max-color-value 5 --max-width-value 5 --min-width-value 0 --max-width 3 --min-width .5 --colormap winter -o density-color-map.png
```

Generación de estadísticas de la simulación en SUMO

4 Práctica 3: Estadísticas de movilidad vehicular en SUMO

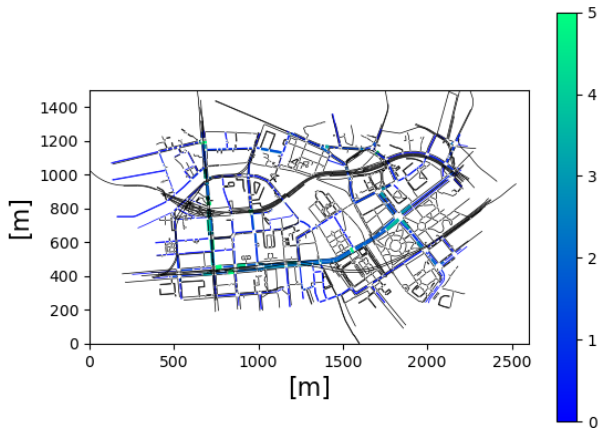


Tabla de contenidos

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

- ▶ Introducción
- ▶ Práctica 1: Introducción al Simulador de movilidad Urbana (SUMO)
- ▶ Práctica 2: Simulación de tráfico vehicular con SUMO WebWizard
- ▶ Práctica 3: Estadísticas de movilidad vehicular en SUMO
- ▶ **Práctica 4: Introducción a TraCI (Traffic Control Interface)**
- ▶ Práctica 5: Clasificación de movilidad con aprendizaje automático
- ▶ Práctica 6: Aprendizaje automático aplicado a ITS

Introducción a TraCI con SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Objetivos

- Introducir la librería traci para python.
- Verificar la variable de entorno de SUMO con TraCI.

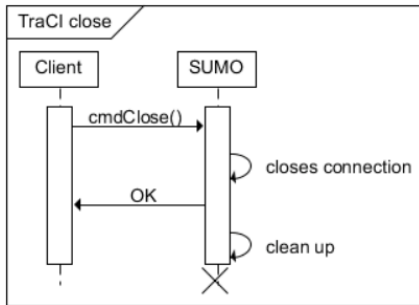
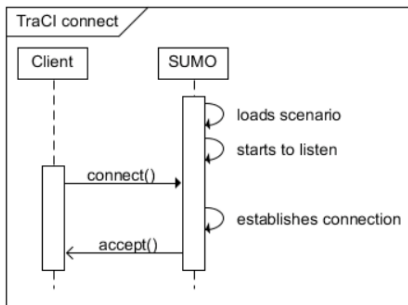
Materiales

- **Ordenador** con sistema operativo Linux (Ubuntu 22.04.3 LTS).
- **SUMO** instalado.

Introducción a TraCI con SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

- TraCI es el término corto para “Interfaz de control de tráfico” (Traffic Control Interface).
- Al dar acceso a una simulación de tráfico en marcha, permite recuperar valores de objetos simulados y manipular su comportamiento “en línea”.
- TraCI utiliza una arquitectura cliente/servidor basada en TCP para proporcionar acceso a sumo mediante el puerto de sumo designado.



Introducción a TraCI con SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Instrucciones

1. El primer paso es instalar la librería traci para python.

```
pip3 install traci
```

2. En este punto solo falta codificar en python las acciones que se requieran realizar. Como primer paso es necesario comprobar que la variable de entorno SUMO_HOME esté definida. En el repositorio se presenta el script para realizar la comprobación de la variable de entorno (traci_check_sumo_home). Disponible en <https://github.com/erickperez9/ManualSUMO.git> en la ubicación Scripts TraCI₅UMO
3. Donde, se importa la librería traci para verificar la ubicación de los archivos traci. Además, se importa las librerías sys y os para comprobar en el sistema operativo la existencia de la variable de entorno.

Introducción a TraCI con SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

```
sumo@sumo-VirtualBox:~/sumo/tools/2024-02-05-16-21-35$ python3 traci_check_sumo_home.py
La variable de entorno SUMO_HOME está definida.
Su valor es: /home/sumo/sumo

LOADPATH: /home/sumo/sumo/tools/2024-02-05-16-21-35
/usr/lib/python310.zip
/usr/lib/python3.10
/usr/lib/python3.10/lib-dynload
/home/sumo/.local/lib/python3.10/site-packages
/usr/local/lib/python3.10/dist-packages
/usr/lib/python3/dist-packages
/home/sumo/sumo/tools
TRACIPATH: /usr/local/lib/python3.10/dist-packages/traci/ init .py
sumo@sumo-VirtualBox:~/sumo/tools/2024-02-05-16-21-35$
```

- Con estos simples pasos, es posible conectar TraCI con SUMO y realizar la codificación para requerimientos específicos, utilizando la librería traci para python. Los comandos pueden lograr lo siguiente:
 - **Valores a recuperar:** Objetos de tráfico, Detectores y Salidas, Red, Infraestructura, Varios
 - **Cambio de estado:** Objetos de tráfico, Detectores y Salidas, Red, Infraestructura, Varios
 - **Suscripciones:** TraCI/Object Variable Subscription, TraCI/Object Context Subscription

Simulación de tráfico vehicular TraCI/SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Objetivos

- Integrar la interfaz TraCI con SUMO.
- Simular una red vehicular sobre SUMO utilizando scripts en python.

Materiales

- **Ordenador** con sistema operativo Linux (Ubuntu 22.04.3 LTS).
- **SUMO** instalado.

Simulación de tráfico vehicular TraCI/SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Introducción

- SUMO es una plataforma de simulación de tráfico de código abierto que permite modelar y analizar el comportamiento de vehículos, peatones y otros agentes en entornos urbanos.
- Dentro del ecosistema de herramientas de SUMO, TraCI desempeña un papel crucial al **permitir la comunicación en tiempo real entre simulaciones externas y el simulador SUMO.**
- **TraCI ofrece a los desarrolladores y usuarios la capacidad de controlar y monitorear de manera dinámica los elementos de la simulación,** como vehículos, semáforos, y otros agentes de tráfico.

Simulación de tráfico vehicular TraCI/SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Instrucciones

1. El primer paso es importar las librerías necesarias. En este caso solo se utilizará `traci`.
2. Luego, es necesario iniciar la comunicación SUMO/TraCI a través del comando `traci.start(['cmd'])`. Donde, `cmd` hace referencia al comando que se ejecutará (`sumo` o `sumo-gui`). Finalmente, se debe indicar la ubicación del archivo de configuración de la simulación de SUMO, con el parámetro `-c`.
3. Luego, es necesario iniciar el bucle de la simulación. Donde, la condición debe indicar la duración que deseamos de la simulación.
4. En el documento *Manual de laboratorio SUMO*, se presenta el script (`traci_run.py`) utilizado para correr una simulación de SUMO, conectando con TraCI.

Simulación de tráfico vehicular TraCI/SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Instrucciones

A continuación vemos el script de Python correspondiente a la lógica del bucle considerando un tiempo estimado de simulación de una hora (3600 segundos) y 3700 pasos:

```
import traci

traci.start(["sumo", "-c", "/home/osm.sumocfg"])
step = 0

while step < 3700:
    traci.simulationStep()
    step += 1

traci.close()
```

Simulación de tráfico vehicular TraCI/SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

```
Simulation ended at time: 3950.00
Reason: TraCI requested termination.
Performance:
  Duration: 9.26s
  TraCI-Duration: 0.70s
  Real time factor: 426.796
  UPS: 8284.062669
Vehicles:
  Inserted: 389
  Running: 0
  Waiting: 0
Statistics (avg of 389):
  RouteLength: 1446.19
  Speed: 7.45
  Duration: 197.09
  WaitingTime: 28.26
  TimeLoss: 60.80
  DepartDelay: 0.58

DijkstraRouter answered 389 queries and explored 354.39 edges on average.
DijkstraRouter spent 0.08s answering queries (0.19ms on average).
sumo@sumo-VirtualBox:~/sumo/tools/2024-02-05-16-21-35$
```

Simulación TraCI/SUMO exitosa.

Simulación de tráfico vehicular TraCI/SUMO. Contador de vehículos

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Objetivos

- Integrar la interfaz TraCI con SUMO.
- Simular una red vehicular sobre SUMO utilizando scripts en python.
- Contar el número de vehículos que circularon en el mapa.

Materiales

- **Ordenador** con sistema operativo Linux (Ubuntu 22.04.3 LTS).
- **SUMO** instalado.

Simulación de tráfico vehicular TraCI/SUMO. Contador de vehículos

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Introducción

- El estudio y la gestión del tráfico urbano son aspectos críticos para el desarrollo de ciudades sostenibles y eficientes.
- En este contexto, SUMO ha surgido como una herramienta poderosa que permite modelar y simular el flujo de vehículos y peatones en entornos urbanos de manera realista.
- Dentro del conjunto de herramientas que ofrece SUMO, desempeña un papel fundamental al facilitar la comunicación en tiempo real entre simulaciones externas y el simulador SUMO.
- Una de las capacidades más destacadas de TraCI es su habilidad para contar y rastrear vehículos dentro de la simulación, lo que brinda a los investigadores y planificadores una perspectiva detallada del flujo de tráfico en entornos urbanos.

Simulación de tráfico vehicular TraCI/SUMO. Contador de vehículos

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Instrucciones

1. Importar las librerías necesarias: `traci` y `re` para el manejo de expresiones regulares.
2. Iniciar la comunicación SUMO/TraCI a través del comando `traci.start(['cmd'])` e iniciar el bucle de la simulación y guardar los identificadores de los vehículos.
3. Dentro del bucle, es necesario extraer los identificadores de los vehículos, esto son una lista y se obtiene con la instrucción `traci.vehicle.getIDList()`.
4. Finalmente, es necesario crear una lógica con la cual se extraiga los números identificadores de cada vehículo y obtener el número mayor. Con esto estaremos obteniendo el número total de vehículos que se movilizaron sobre el mapa.
5. En el documento *Manual de laboratorio SUMO*, se presenta el script (`traci_cont_veh.py`) utilizado para correr una simulación de SUMO, conectando con TraCI.

Simulación de tráfico vehicular TraCI/SUMO. Contador de vehículos

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

```
Simulation ended at time: 3950.00
Reason: TraCI requested termination.
Performance:
  Duration: 9.70s
  TraCI-Duration: 1.24s
  Real time factor: 407.427
  UPS: 7908.096957
```

```
Vehicles:
  Inserted: 389
  Running: 0
  Waiting: 0
Statistics (avg of 389):
  RouteLength: 1446.19
  Speed: 7.45
  Duration: 197.09
  WaitingTime: 28.26
  TimeLoss: 60.80
  DepartDelay: 0.58
```

```
DijkstraRouter answered 389 queries and explored 354.39 edges on average.
DijkstraRouter spent 0.09s answering queries (0.23ms on average).
```

En el mapa han circulado un total de 429 vehículos

```
sumo@sumo-VirtualBox:~/sumo/tools/2024-02-05-16-21-35$
```


Simulación de tráfico vehicular TraCI/SUMO. Cálculo de velocidad promedio

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Objetivos

- Integrar la interfaz TraCI con SUMO.
- Simular una red vehicular sobre SUMO utilizando scripts en python.
- Calcular la velocidad promedio de los vehículos que circularon en el mapa.

Materiales

- **Ordenador** con sistema operativo Linux (Ubuntu 22.04.3 LTS).
- **SUMO** instalado.

Simulación de tráfico vehicular TraCI/SUMO. Cálculo de velocidad promedio

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Introducción

- La simulación del tráfico urbano es fundamental para comprender y optimizar la movilidad en entornos urbanos cada vez más complejos.
- Dentro del conjunto de herramientas proporcionadas por SUMO, TraCI ofrece una gama de funcionalidades, entre las cuales destaca la capacidad para calcular y registrar las velocidades de los vehículos dentro de la simulación.

Simulación de tráfico vehicular TraCI/SUMO. Cálculo de velocidad promedio

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Instrucciones

1. Importar las librerías necesarias. En este caso se utilizará `traci`, `csv` y `time`.
2. Iniciar la comunicación SUMO/TraCI a través del comando `traci.start(['cmd'])`.
3. Indicar la ruta del archivo de salida, abrir el archivo csv.
4. Iniciar el bucle de la simulación para guardar los identificadores de los vehículos, velocidades, tiempos de simulación y verificar el nombre de calle (borde) en el que se encuentra. En este caso se los datos se guardan en un archivo de salida de tipo csv.
5. Para obtener el tiempo de simulación en segundos se usa:
`traci.simulation.getCurrentTime() / 1000`
6. Para obtener los IDs se utiliza:
`traci.vehicle.getIDList()`

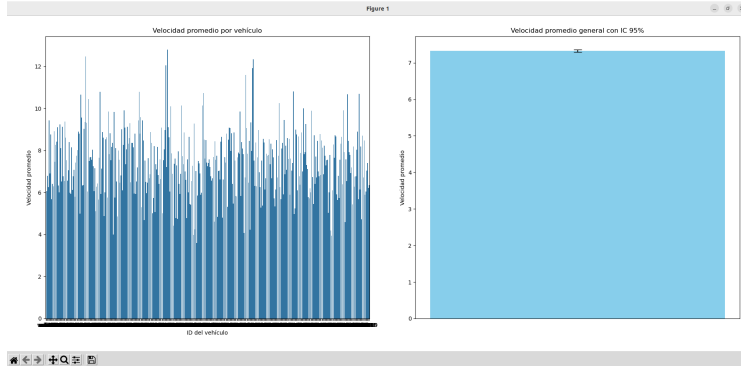
Simulación de tráfico vehicular TraCI/SUMO. Cálculo de velocidad promedio

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

7. Para obtener la velocidad se utiliza:
`traci.vehicle.getSpeed(veh_id)`
8. Para obtener la calle (edge) actual se utiliza:
`traci.vehicle.getRoadID(veh_id)`
9. Finalmente, es necesario crear una lógica con la cual se verifique la velocidad del vehículo. Si la velocidad es menor a 10 m/s, el destino del vehículo cambia. Por ejemplo, el vehículo con ID `veh424` y se lo redirige al destino con ID `234463928#1`. Los IDs de los bordes de las carreteras se pueden verificar en los archivos de configuración pertinentes.
10. En el documento *Manual de laboratorio SUMO*, se presenta el script (`traci_vel.py`) utilizado para correr una simulación de SUMO, conectando con TraCI.

Simulación de tráfico vehicular TraCI/SUMO. Cálculo de velocidad promedio

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)



Cálculo de velocidades promedio con TraCI/SUMO.

Simulación de tráfico vehicular TraCI/SUMO. Cálculo de emisiones CO₂

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Objetivos

- Integrar la interfaz TraCI con SUMO.
- Simular una red vehicular sobre SUMO utilizando scripts en python.
- Calcular las emisiones de CO₂ de los vehículos que circularon en el mapa.

Materiales

- **Ordenador** con sistema operativo Linux (Ubuntu 22.04.3 LTS).
- **SUMO** instalado.

Simulación de tráfico vehicular TraCI/SUMO. Cálculo de emisiones CO₂

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Introducción

- Dentro del conjunto de herramientas que ofrece SUMO, TraCI desempeña un papel crucial al permitir la interacción en tiempo real entre simulaciones externas y el simulador SUMO.
- Una de las capacidades más destacadas de TraCI es su capacidad para calcular y registrar las emisiones de dióxido de carbono (CO₂) de los vehículos en la simulación.

Simulación de tráfico vehicular TraCI/SUMO. Cálculo de emisiones CO₂

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Instrucciones

1. Importar las librerías necesarias. En este caso se utilizará: `traci`, `csv`, `time`, `panda`, `matplotlib` y `seaborn`, `stats`.
2. Iniciar la comunicación SUMO/TraCI a través del comando `traci.start(['cmd'])`.
3. Indicar la ruta del archivo de salida, abrir el archivo csv e iniciar el bucle de la simulación para guardar los identificadores de los vehículos, emisiones CO₂ y tiempos de simulación. En este caso se los datos se guardan en un archivo de salida de tipo csv.
4. Para obtener el tiempo de simulación en segundos se usa:
`traci.simulation.getCurrentTime() / 1000`

Simulación de tráfico vehicular TraCI/SUMO. Cálculo de emisiones CO₂

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

5. Para obtener los IDs se utiliza:

```
traci.vehicle.getIDList()
```

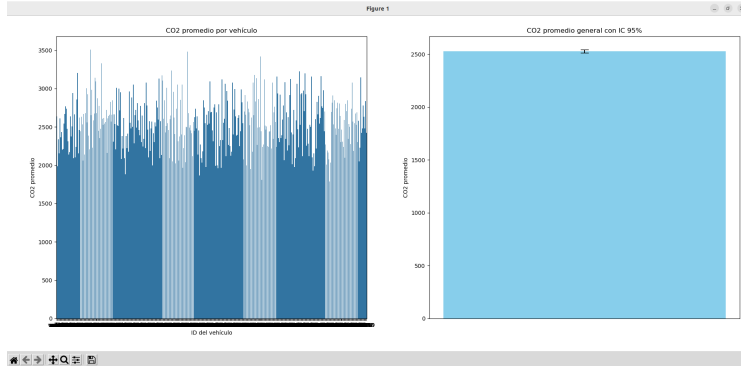
6. Para obtener las emisiones de CO₂ se utiliza:

```
traci.vehicle.getCO2Emission(veh_id)
```

7. Finalmente, es necesario crear una lógica con la cual lee la información del archivo de salida csv, para luego obtener las emisiones (mg/s) media de cada vehículo y de la simulación en general. Finalmente, se gráfica las emisiones de CO₂ obtenidas.
8. En el documento *Guia de laboratorio SUMO*, se presenta el script (`traci_co2.py`) utilizado para correr una simulación de SUMO, conectando con TraCI.

Simulación de tráfico vehicular TraCI/SUMO. Cálculo de emisiones CO₂

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)



Cálculo de emisiones CO₂ promedio con TraCI/SUMO.

Simulación de tráfico vehicular TraCI/SUMO. Cambio de trayectoria de vehículos

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Objetivos

- Integrar la interfaz TraCI con SUMO.
- Simular una red vehicular sobre SUMO utilizando scripts en python.
- Cambiar la trayectoria de los vehículos que circularon en el mapa, en base a una lógica dada.

Materiales

- **Ordenador** con sistema operativo Linux (Ubuntu 22.04.3 LTS).
- **SUMO** instalado.

Simulación de tráfico vehicular TraCI/SUMO. Cambio de trayectoria de vehículos

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Introducción

- Dentro del conjunto de herramientas disponibles en SUMO, TraCI proporciona una interfaz dinámica que permite la interacción en tiempo real con la simulación de tráfico.
- Una de las características más destacadas de TraCI es su capacidad para calcular nuevas rutas para los vehículos en función de circunstancias específicas, como las velocidades de los vehículos en las calles.

Simulación de tráfico vehicular TraCI/SUMO. Cambio de trayectoria de vehículos

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Instrucciones

1. Importar las librerías necesarias. En este caso se utilizará: `traci`, `re`, `csv`, `time`, `panda`, `matplotlib` y `seaborn`, `stats`.
2. Iniciar la comunicación SUMO/TraCI a través del comando `traci.start(['cmd'])`.
3. Indicar la ruta del archivo de salida, abrir el archivo csv e iniciar el bucle de la simulación para guardar los identificadores de los vehículos, emisiones CO2 y tiempos de simulación.
4. Para obtener el tiempo de simulación en segundos se usa:
`traci.simulation.getCurrentTime() / 1000`

Simulación de tráfico vehicular TraCI/SUMO. Cambio de trayectoria de vehículos

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

5. Para obtener los IDs se utiliza:

```
traci.vehicle.getIDList()
```

6. Para obtener la velocidad se utiliza:

```
traci.vehicle.getSpeed(veh_id)
```

7. Para obtener el ID de la calle (edge) se utiliza:

```
traci.vehicle.getRoadID(veh_id)
```

8. Para cambiar el destino de un vehículo se utiliza:

```
traci.vehicle.changeTarget(veh_id,EDGE_ID)
```

9. Crear una lógica con la cual se lea la velocidad del vehículo en análisis y poder así re-enrutar su trayectoria hacia un punto en específico. En el documento *Manual de laboratorio SUMO* se indica el script `traci_change_pos` con el que se logra esto.

Simulación de tráfico vehicular TraCI/SUMO. Cambio de trayectoria de vehículos

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

```
Tiempo: 3904.0, ID del Vehículo: veh424, Velocidad: 15.803129615470938, EdgeNow: :2425782615_5
**** SE HA CAMBIADO LA RUTA DEL VEHICULO veh424 HACIA 234463928#1
Tiempo: 3905.0, ID del Vehículo: veh424, Velocidad: 15.220921826431699, EdgeNow: :1275468904_1
**** SE HA CAMBIADO LA RUTA DEL VEHICULO veh424 HACIA 234463928#1
Tiempo: 3906.0, ID del Vehículo: veh424, Velocidad: 15.466820430428893, EdgeNow: 234463928#1
**** SE HA CAMBIADO LA RUTA DEL VEHICULO veh424 HACIA 234463928#1
Tiempo: 3907.0, ID del Vehículo: veh424, Velocidad: 14.763622594298035, EdgeNow: 234463928#1
Simulation ended at time: 3908.00
Reason: TraCI requested termination.
Performance:
  Duration: 24.14s
  TraCI-Duration: 10.57s
  Real time factor: 161.916
  UPS: 3174.801127
Vehicles:
  Inserted: 389
  Running: 0
  Waiting: 0
Statistics (avg of 389):
  RouteLength: 1445.23
  Speed: 7.45
  Duration: 196.98
  WaitingTime: 28.26
  TimeLoss: 60.79
  DepartDelay: 0.58
```

Destino cambiado para vehículo veh424.

NetEdit - Editor de mapas de SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

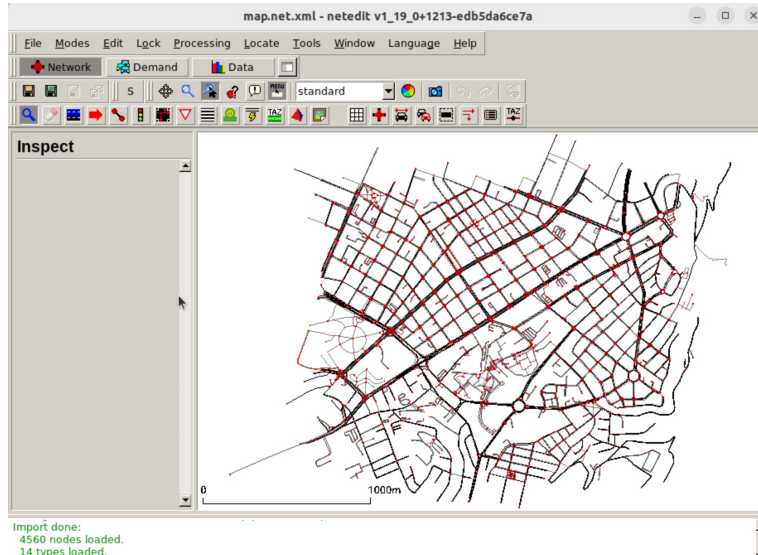
- Netedit es un **editor visual de la red** de carreteras.
- Se puede utilizar para **crear redes desde cero**
- Se puede utilizar para **depurar atributos** de red.
- **Modificar** todos los aspectos de las redes existentes.

netedit

Documentación: <https://sumo.dlr.de/docs/Netedit/index.html>

NetEdit - Editor de mapas de SUMO

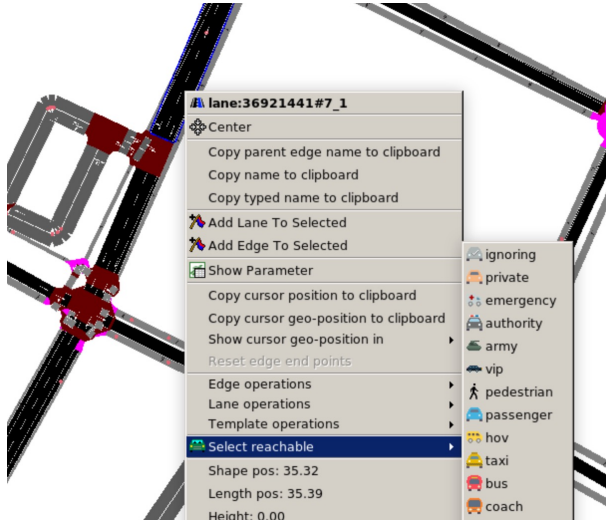
5 Práctica 4: Introducción a TraCI (Traffic Control Interface)



NetEdit - Editor de mapas de SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

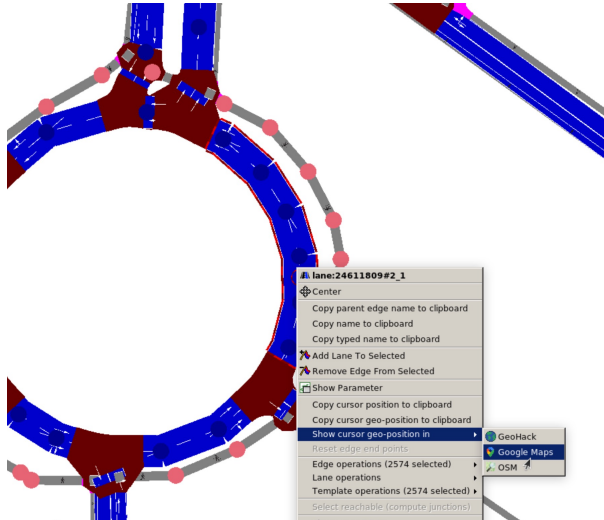
Identificación y modificación de carreteras:



NetEdit - Editor de mapas de SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

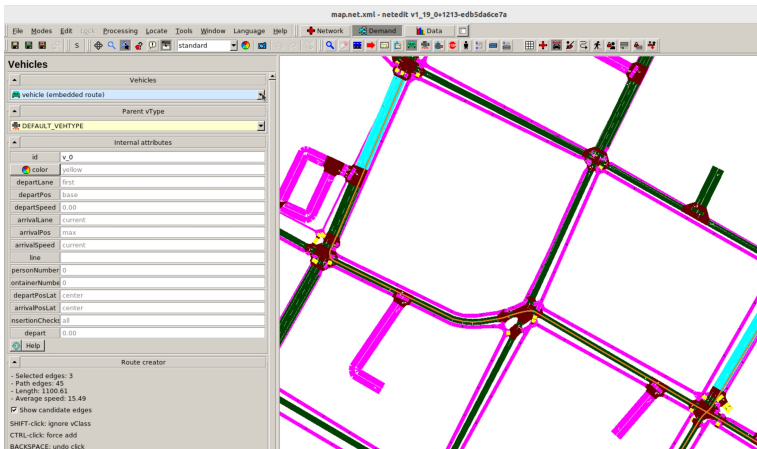
Identificación y modificación de carreteras (Google Maps):



NetEdit - Editor de mapas de SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

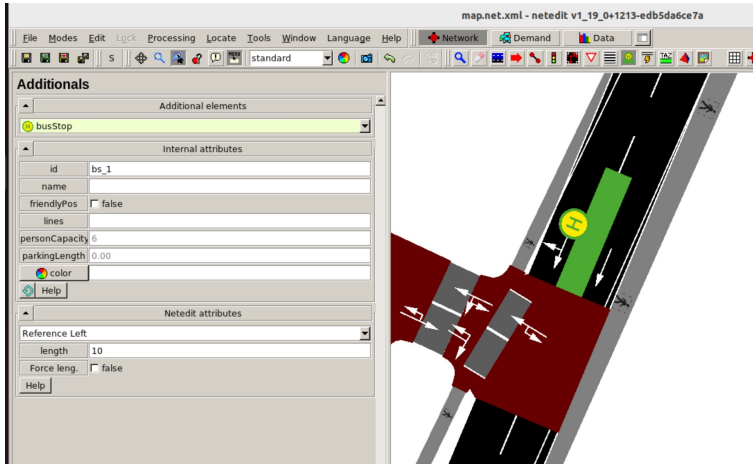
Rutas de transporte público:



NetEdit - Editor de mapas de SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

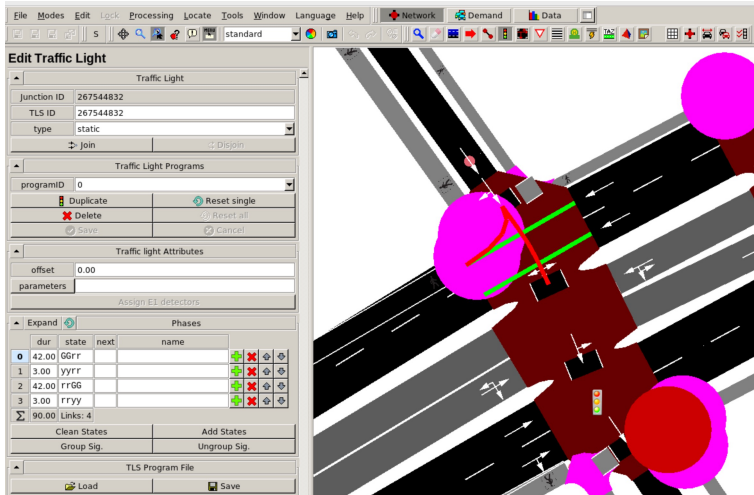
Parada de Bus:



NetEdit - Editor de mapas de SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Rutas de transporte público:



NetEdit - Editor de mapas de SUMO

5 Práctica 4: Introducción a TraCI (Traffic Control Interface)

Traffic Assignment Zone:

